

Lecture – 7: Newton Divide Difference Formula Derivation

Setting the stage

Suppose we already have a degree- n polynomial $p_n(x)$ that interpolates f at the nodes $x_0, x_1, \dots, x_n$:

$$f(x_k) = p_n(x_k), \quad k = 0, 1, \dots, n.$$

We now want to add *one new node* x_{n+1} and build $p_{n+1}(x)$ — but without throwing away $p_n(x)$ and starting over. Newton's idea: build p_{n+1} by *correcting* p_n with one extra term.

1 The correction term $g_{n+1}(x)$

We write

$$p_{n+1}(x) = p_n(x) + g_{n+1}(x).$$

Think of $g_{n+1}(x)$ as a patch: it must fix the value at the brand-new node x_{n+1} , while leaving everything p_n already got right completely untouched:

$$\underbrace{p_{n+1}(x)}_{\text{fits } x_0, \dots, x_n, x_{n+1}} = \underbrace{p_n(x)}_{\text{already fits } x_0, \dots, x_n} + \underbrace{g_{n+1}(x)}_{\text{the patch}}.$$

For this patch to work, it needs exactly two properties.

Property 1 - Degree. p_{n+1} has degree $n + 1$ and p_n has degree n , so $g_{n+1}(x)$ must supply the missing degree: it must be a degree- $(n + 1)$ polynomial.

Property 2 - Do no harm. At the old nodes, p_n is already exactly correct: $p_n(x_k) = f(x_k)$ for $k = 0, \dots, n$. Adding $g_{n+1}(x)$ must not disturb that, so we need $g_{n+1}(x_k) = 0$ for $k = 0, 1, \dots, n$. In other words, $x_0, x_1, \dots, x_n$ must all be *roots* of $g_{n+1}(x)$.

Why this works

A degree- $(n + 1)$ polynomial with $(n + 1)$ prescribed roots $x_0, \dots, x_n$ can only take one shape (the factor theorem): a constant times $(x - x_0)(x - x_1) \cdots (x - x_n)$. There is no other possibility.

So:

$$g_{n+1}(x) = c_{n+1} (x - x_0)(x - x_1) \cdots (x - x_n)$$

for some constant c_{n+1} . This single formula automatically satisfies *both* properties at once: it vanishes at every old node (Property 2), and being built from $(n + 1)$ linear factors, it has degree exactly $n + 1$ (Property 1).

c_{n+1} is called **Newton's coefficient** - the one unknown number left to pin down.

1.1 Newton's basis polynomials

The product pattern above is useful enough to deserve its own notation. Define

$$n_k(x) \equiv (x - x_0)(x - x_1) \cdots (x - x_{k-1}) = \prod_{j=0}^{k-1} (x - x_j), \quad k = 1, 2, \dots, n, \quad n_0(x) \equiv 1.$$

These are the **Newton basis polynomials**:

$$\begin{aligned} n_0(x) &= 1, & n_1(x) &= (x - x_0), & n_2(x) &= (x - x_0)(x - x_1), \\ & \dots, & n_n(x) &= (x - x_0)(x - x_1) \cdots (x - x_{n-1}). \end{aligned}$$

Each $n_k(x)$ has degree k and vanishes at exactly $x_0, \dots, x_{k-1}$ - Property 2, one level down. With this notation,

$$g_{n+1}(x) = c_{n+1} n_{n+1}(x).$$

2 Solving for the coefficient: the residual idea

We still need c_{n+1} . We get it by demanding that p_{n+1} actually agrees with f at the new node. At $x = x_{n+1}$:

$$f(x_{n+1}) = p_{n+1}(x_{n+1}) = p_n(x_{n+1}) + g_{n+1}(x_{n+1}) \implies g_{n+1}(x_{n+1}) = f(x_{n+1}) - p_n(x_{n+1}).$$

What this really means

g_{n+1} evaluated at the new node equals the *error* (residual) that the old polynomial p_n makes there — the gap between the true value $f(x_{n+1})$ and what the old approximation predicts. That is the precise meaning behind the shorthand “ $g_{n+1}(x) = f(x) - p_n(x)$ ”: it is the residual, evaluated at the node where the new condition is imposed - not an identity that holds for every x .

Combining with $g_{n+1}(x_{n+1}) = c_{n+1}(x_{n+1} - x_0)(x_{n+1} - x_1) \cdots (x_{n+1} - x_n)$:

$$c_{n+1} = \frac{f(x_{n+1}) - p_n(x_{n+1})}{(x_{n+1} - x_0)(x_{n+1} - x_1) \cdots (x_{n+1} - x_n)}.$$

Re-indexing ($n + 1 \rightarrow k$) gives the pattern for *any* step:

$$c_k = \frac{f(x_k) - p_{k-1}(x_k)}{(x_k - x_0)(x_k - x_1) \cdots (x_k - x_{k-1})}, \quad k = 0, 1, \dots, n,$$

with the convention $p_{-1}(x) \equiv 0$, so that $c_0 = f(x_0)$.

3 The Newton form of $p_n(x)$

Stacking every correction term together,

$$p_n(x) = \sum_{k=0}^n c_k n_k(x) = c_0 n_0(x) + c_1 n_1(x) + \cdots + c_n n_n(x),$$

and substituting what each $n_k(x)$ actually is, term by term:

$$p_n(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \cdots + c_n(x - x_0)(x - x_1) \cdots (x - x_{n-1}).$$

This is the **Newton form** of the interpolating polynomial. Its nice feature: going from p_n to p_{n+1} just tacks on one more term at the end - nothing already written down needs to change.

4 From Newton's coefficients to Divided Differences

The boxed formula for c_k above still has $p_{k-1}(x_k)$ inside it - another polynomial we would have to build and evaluate. We can instead get a fully explicit formula for $c_0, c_1, c_2, \dots$ directly, by substituting specific nodes into the Newton form of $p_n(x)$ itself, since we already know $p_n(x_k) = f(x_k)$ at every node.

Setting $x = x_0$

Every term in $p_n(x)$ from c_1 onward contains a factor $(x - x_0)$, which is zero when $x = x_0$:

$$p_n(x_0) = c_0 + \underbrace{c_1(x_0 - x_0)}_0 + \underbrace{c_2(x_0 - x_0)(x_0 - x_1)}_0 + \cdots = c_0.$$

Since $p_n(x_0) = f(x_0)$:

$$c_0 = f(x_0) \equiv f[x_0].$$

The square-bracket notation $f[x_0]$ is just another name for $f(x_0)$: the **zeroth-order divided difference**.

Setting $x = x_1$

Now every term from c_2 onward contains a factor $(x - x_1)$, zero at $x = x_1$:

$$p_n(x_1) = c_0 + c_1(x_1 - x_0) + \underbrace{c_2(x_1 - x_0)(x_1 - x_1)}_0 + \cdots = c_0 + c_1(x_1 - x_0).$$

Since $p_n(x_1) = f(x_1) \equiv f[x_1]$, and using $c_0 = f[x_0]$:

$$f[x_1] = f[x_0] + c_1(x_1 - x_0) \implies c_1 = \frac{f[x_1] - f[x_0]}{x_1 - x_0} \equiv f[x_0, x_1].$$

This is the **first-order divided difference**: literally a slope (rise over run) between the points $(x_0, f(x_0))$ and $(x_1, f(x_1))$.

Setting $x = x_2$ - the one worth slowing down for

Every term from c_3 onward vanishes (each carries a factor $(x - x_2)$), leaving

$$p_n(x_2) = c_0 + c_1(x_2 - x_0) + c_2(x_2 - x_0)(x_2 - x_1) = f(x_2) \equiv f[x_2].$$

Substituting the values of c_0 and c_1 already found:

$$f[x_2] = f[x_0] + f[x_0, x_1](x_2 - x_0) + c_2(x_2 - x_0)(x_2 - x_1),$$

$$c_2 = \frac{f[x_2] - f[x_0] - f[x_0, x_1](x_2 - x_0)}{(x_2 - x_0)(x_2 - x_1)}. \quad (*)$$

This is correct, but it doesn't yet look like the pattern we expect: a difference of two divided differences, over $(x_2 - x_0)$. Reaching that clean form takes exactly one trick - *insert and immediately subtract* $f[x_1]$ in the numerator of $(*)$. Two small facts are used along the way, both direct rearrangements of definitions already established:

- **Fact A:** $f[x_1] - f[x_0] = f[x_0, x_1](x_1 - x_0)$ (rearranging $c_1 = f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0}$)
- **Fact B:** $f[x_2] - f[x_1] = f[x_1, x_2](x_2 - x_1)$ (same first-order divided-difference idea, one node to the right)

Now the numerator of $(*)$, step by step:

1. Insert $+f[x_1] - f[x_1]$ (adds zero, changes nothing):

$$f[x_2] - f[x_0] - f[x_0, x_1](x_2 - x_0) = (f[x_2] - f[x_1]) + (f[x_1] - f[x_0]) - f[x_0, x_1](x_2 - x_0).$$

2. Replace $f[x_1] - f[x_0]$ using **Fact A**:

$$= (f[x_2] - f[x_1]) + f[x_0, x_1](x_1 - x_0) - f[x_0, x_1](x_2 - x_0).$$

3. Factor $f[x_0, x_1]$ out of the last two terms:

$$= (f[x_2] - f[x_1]) + f[x_0, x_1] \left[(x_1 - x_0) - (x_2 - x_0) \right].$$

4. Simplify the bracket: $(x_1 - x_0) - (x_2 - x_0) = x_1 - x_2 = -(x_2 - x_1)$, so:

$$= (f[x_2] - f[x_1]) - f[x_0, x_1](x_2 - x_1).$$

5. Replace $f[x_2] - f[x_1]$ using **Fact B**, then factor out $(x_2 - x_1)$:

$$= (x_2 - x_1) f[x_1, x_2] - (x_2 - x_1) f[x_0, x_1] = (x_2 - x_1) (f[x_1, x_2] - f[x_0, x_1]).$$

So the numerator of $(*)$ equals $(x_2 - x_1)(f[x_1, x_2] - f[x_0, x_1])$. Dividing by the denominator $(x_2 - x_0)(x_2 - x_1)$, the $(x_2 - x_1)$ factor cancels:

$$c_2 = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} \equiv f[x_0, x_1, x_2].$$

This is the **second-order divided difference**: a “slope of slopes” - the rate of change between the two first-order divided differences $f[x_0, x_1]$ and $f[x_1, x_2]$.

Setting $x = x_3$ - confirming the pattern continues

It is worth checking, at least once, that the same machinery keeps working. Every term from c_4 onward vanishes at $x = x_3$ (factor $(x - x_3)$), leaving

$$p_n(x_3) = c_0 + c_1(x_3 - x_0) + c_2(x_3 - x_0)(x_3 - x_1) + c_3(x_3 - x_0)(x_3 - x_1)(x_3 - x_2) = f[x_3].$$

Substituting the known c_0, c_1, c_2 and solving for c_3 gives a raw expression exactly analogous to (*). Applying the *same* insert-and-subtract trick as before — this time inserting $+f[x_1, x_2] - f[x_1, x_2]$ instead of $\pm f[x_1]$, and using $f[x_1, x_2, x_3] := \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1}$ in place of Fact B - collapses it the same way:

$$c_3 = \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0} \equiv f[x_0, x_1, x_2, x_3].$$

So the trick is not a one-off coincidence at $k = 2$ — it is a genuine, repeatable pattern: substitute the already-known coefficients, insert-and-subtract the right lower-order divided difference, factor, and cancel.

5 The general pattern

Continuing the same substitution for every remaining node ($x = x_4, x = x_5, \dots$) reproduces the same style of algebra every time, always collapsing to a difference of two divided differences over a node gap. The general recursive rule is:

$$f[x_0, x_1, \dots, x_{k-1}, x_k] = \frac{f[x_1, \dots, x_{k-1}, x_k] - f[x_0, x_1, \dots, x_{k-1}]}{x_k - x_0},$$

and it holds for a divided difference over *any* contiguous block of nodes, not only those starting at x_0 :

$$f[x_i, x_{i+1}, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

So the Newton coefficients are exactly the divided differences of f :

$$c_0 = f[x_0], \quad c_1 = f[x_0, x_1], \quad c_2 = f[x_0, x_1, x_2], \quad \dots, \quad c_n = f[x_0, x_1, \dots, x_n].$$

This recursive structure is exactly what makes the **Divided-Difference table** work: each new column is just a difference of two entries in the previous column, divided by the gap between the outer nodes.

6 Quick numeric check

Take three nodes $x_0 = 0, x_1 = 1, x_2 = 3$ with $f(x_0) = 1, f(x_1) = 2, f(x_2) = 10$.

$$f[x_0, x_1] = \frac{2 - 1}{1 - 0} = 1, \quad f[x_1, x_2] = \frac{10 - 2}{3 - 1} = 4, \quad f[x_0, x_1, x_2] = \frac{4 - 1}{3 - 0} = 1.$$

Checking directly against (*):

$$c_2 = \frac{10 - 1 - (1)(3 - 0)}{(3 - 0)(3 - 1)} = \frac{10 - 1 - 3}{6} = \frac{6}{6} = 1. \quad \checkmark$$

Same answer, both routes.

Glossary of terms

Symbol	Name	Meaning
$p_n(x)$	Interpolating polynomial	degree- n polynomial matching f at $x_0, \dots, x_n$
$g_{n+1}(x)$	Correction term	degree- $(n + 1)$ patch turning p_n into p_{n+1}
$n_k(x)$	Newton basis polynomial	$\prod_{j=0}^{k-1} (x - x_j)$
c_k	Newton's coefficient	weight of $n_k(x)$ in the Newton form of $p_n(x)$
$f[x_0, \dots, x_k]$	k -th order divided difference	equals c_k ; defined recursively from lower orders

Next up: these divided differences get arranged into a triangular table (the Divided-Difference table) so all the c_k 's for a full data set can be read off in one pass, without repeating this derivation by hand each time.